Evaluating Streaming and In-Memory CSV Approaches for Data Ingestion and Querying

Brandon Llanes
Meghana Koti

Introduction

The NYC Yellow Taxi dataset contains over 30 million records per year across 18 columns. To meet the required data size, datasets from 2019–2022 were combined, totaling 24.1 GB.

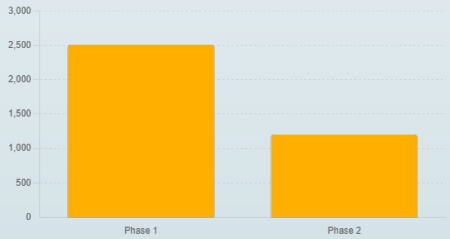
The goal was to build an application to load the data and execute subset queries. The project was structured in three phases: a serial baseline, a parallel implementation, and a redesigned data model.

To evaluate trade-offs, both streaming and in-memory approaches were implemented and compared across all phases.

In-Memory Ingestion Time Comparison (Phase ½)

An in-memory approach has an upfront ingestion cost. Using OpenMP threading, we achieve a speedup of **2.09x**. However, it is bounded by CPU-bound datetime parsing and I/O bandwidth.

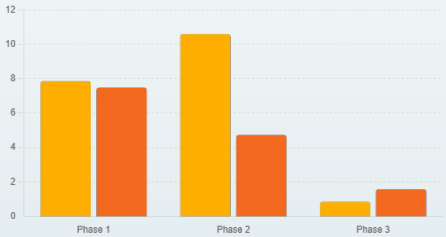
Ingestion Performance



In-Memory Approach

Query time in phase 2 increases due to resource contention since workload is memory-bandwidth bound. Threads process fewer rows per second.

Query Performance



Threads process their contiguous index regions, only necessary columns access, and fewer cache lines touched.

Row Throughput

Streaming Approach

Single-pass streaming design that processes CSV files line-by-line using reusable record objects, with file-level OpenMP parallelism and bounded memory usage.

Key Insights :

1. The workload was **compute-bound**, with parsing dominating execution time.

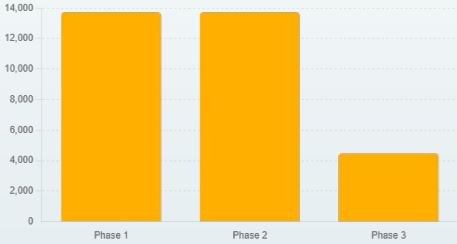
2. File-level parallelism did not reduce per-row parsing cost, leading to negative scaling.

3. Memory-efficient streaming does not automatically translate to scalable performance.

Could be improved : Optimize parser efficiency and apply chunk-based parallel parsing to better utilize threads.

Estimated Data Scanned Per Query

During queries in phase 3, performance is improved as there is no need to scan an entire TaxiTripRecord object. Only column vectors affecting the query subset are scanned.



Conclusion

Across all phases, the following conclusions emerge:

1) Streaming is efficient for one-time scans but does not benefit from repeated queries.
2) In-memory execution is initially expensive but provides dramatically faster query performance once data is loaded.
3) Parallelization alone does not guarantee speedup, especially in memory-bound workloads.
4) Memory layout has a larger impact on performance than threading alone.

For large-scale data processing, memory layout and data access patterns often dominate performance more than parallelization alone.